Fase 0 — Definición del problema, la solución y el diseño experimental

> Estado: **PROPUESTA** — pendiente de aprobación por P1, P2, P3 y P4. Los valores marcados con 🔧 se ajustan en la reunión según los equipos reales. Nadie programa ni mide hasta que este documento esté aprobado.

---

1. Planteamiento del problema (redacta: P4)

Contexto. Cuando un programa crea varios hilos o procesos, el sistema operativo debe decidir cuáles se ejecutan sobre los procesadores lógicos disponibles. El efecto sobre el tiempo y el uso de CPU depende del hardware, del scheduler y del tipo de carga.

Problema. Calcular una suma de verificación (checksum) de 64 bits sobre un rango grande de enteros `[0, N)`. Es una carga **CPU-bound sintética**: no usa disco ni red, así que el tiempo refleja la CPU y la planificación, no la entrada/salida.

Pregunta central.

> ¿Cómo cambian el tiempo de ejecución y la utilización de CPU al repartir esa misma carga entre 1, 2, 4, 8 y 16 hilos, o entre varios procesos, en Windows y Linux, y cómo se explica con el scheduler y los procesadores lógicos?

Hipótesis (se contrastan en el informe).

- H1. Con varios LP disponibles, el tiempo baja al aumentar hilos, pero **no de forma lineal**.
- H2. Con afinidad a **1 LP**, más hilos **no reducen el tiempo** (hay concurrencia, no paralelismo).
- H3. Con más hilos que LP, el tiempo **deja de mejorar** y puede empeorar levemente.
- H4. Hilos y procesos obtienen tiempos similares en carga CPU-bound; la diferencia está en el costo de creación y la memoria.
- H5. El scheduler **puede migrar** un hilo entre LP durante su ejecución.

---

2. Solución: procesos requeridos e hilos por proceso (redacta: P1)

El rango `[0, N)` se reparte entre todos los hilos. Cada hilo calcula una suma parcial y el resultado final es la suma de los parciales. Como la suma es conmutativa, **el resultado debe ser idéntico en todas las configuraciones**; esa es la prueba de que el reparto es correcto.

Escenario	Procesos	Hilos por proceso	Total de hilos	Propósito
E0 Secuencial	1	1	1	Línea base
E1 Multihilo	1	1, 2, 4, 8, 16	1–16	Escalamiento y paralelismo
E1-C Multihilo con 1 LP	1	1, 2, 4, 8	1–8	Concurrencia (afinidad a 1 LP)
E2 Multiproceso	1, 2, 4, 8	1	1–8	Hilos vs procesos (mismo N)
E3 Mixto	2	**A: 5 hilos, B: 3 hilos**	8	Hilos distintos por proceso

Roles dentro del programa.

- **Proceso padre (coordinador):** lee parámetros, reparte rangos, crea hijos (si aplica), espera, combina resultados y mide.
- **Proceso hijo (trabajador):** recibe un sub-rango, crea sus hilos, calcula y entrega su parcial.
- **Hilo:** calcula la suma de su sub-rango y registra en qué LP corre.

APIs.

Acción	Linux	Windows
Crear hilo	`pthread_create`	`CreateThread`
Esperar hilo	`pthread_join`	`WaitForSingleObject`
Crear proceso	`fork`	`CreateProcess` (relanza el `.exe` con `--child`)
Esperar proceso	`waitpid`	`WaitForSingleObject`
LP actual	`sched_getcpu`	`GetCurrentProcessorNumber`

🔧 Si un equipo tiene menos de 16 LP, el nivel 16 se mantiene igual: sirve precisamente para estudiar el caso **hilos > LP**.

---

3. Carga de trabajo (redacta: P1; valida: P2)

Kernel (idéntico en C++ y Python):

```
f(i) = ((i mod 97) × (i mod 53)) XOR (i << 3)
resultado = Σ f(i)  (módulo 2^64)
```

Parámetro	Valor
Workload C++ 🔧	Calibrar para que E0 dure **5–10 s** (inicio: 2 000 000 000)
Workload Python 🔧	~10–20 M (el kernel es ~100× más lento)
Compilación	`g++ -std=c++17 -O0 -pthread` (Linux) / `g++ -std=c++17 -O0` (Windows) — **mismas banderas**
Evitar optimización	Acumulador `volatile` + `-O0`

Calibración (se hace una vez, en cada equipo).

1. Correr E0 con 100 M y medir el tiempo.
2. Escalar linealmente hasta ~5–10 s.
3. Confirmar que al **duplicar el workload se duplica el tiempo**; si no, la carga está siendo optimizada.
4. Fijar el valor y **no cambiarlo** durante todas las pruebas.

---

4. Diseño experimental (redacta: P3)

Variables.

Tipo	Variables
Independientes	Modo, N.º de hilos, N.º de procesos, afinidad (1 LP / todos), SO
Dependientes	Tiempo (s), CPU total (%), CPU/LP observado, migraciones, resultado numérico
Controladas	Workload, banderas de compilación, plan de energía, aplicaciones abiertas, equipo conectado a corriente

Matriz de corridas (por sistema operativo, 5 repeticiones cada una).

Bloque	Configuraciones	Corridas
E0	1	5
E1	1, 2, 4, 8, 16 hilos (todos los LP)	25
E1-C	1, 2, 4, 8 hilos (afinidad 1 LP)	20
E2	1, 2, 4, 8 procesos	20
E3	5+3 hilos	5
Total		**75 por SO** (~10–15 min cada SO con workload calibrado)
Python (opcional)	`threading` y `multiprocessing`: 1, 2, 4, 8	40 por SO

Protocolo (obligatorio).

1. Cerrar aplicaciones innecesarias; equipo conectado y con plan de alto rendimiento.
2. Una corrida de calentamiento que se **descarta** antes de cada bloque.
3. Mantener el mismo orden de bloques en ambos SO.

4. Registrar la hora de inicio y la temperatura/estado del equipo si es relevante.
5. Detener si el equipo muestra comportamiento anormal.

Criterios de validez.

- El **resultado numérico es igual** en todas las configuraciones del mismo workload.
- Desviación entre repeticiones **< 10 %**; si es mayor, repetir esa configuración.
- Si E0 dura menos de 3 s, subir el workload.

---

5. Métricas y formato de datos (redacta: P3)

CSV v2 (una línea por corrida; lo imprime el programa con `--csv`):

```csv
language,os,mode,processes,threads_per_process,total_workers,lp_affinity,workload,run,time_seconds,cpu_total_pct,resul
```

Cómo se mide cada cosa.

Dato	Fuente
Tiempo	El programa (`steady_clock`)
CPU total (%)	El programa (`getrusage` / `GetProcessTimes`), contrastado con la herramienta del SO
LP y migraciones por hilo	El programa con `--log-lp`
Afinidad	`taskset -c 0` (Linux), `start /affinity 1` (Windows)
Scheduler	`top`, `htop`, `ps -eLo pid,tid,psr,comm`, `pidstat -t -p PID 1` / Administrador de tareas, Monitor de recursos
Hardware	`lscpu`, `lscpu -e`, `nproc` / `Get-CimInstance Win32_Processor` y `Win32_ComputerSystem`

Derivadas (las calcula P4). Promedio, desviación estándar, rango, speedup `S = T1/TN`, eficiencia `E = S/N`.

6. Caracterización de los equipos (se llena en la reunión)

Dato	Equipo Windows	Equipo Linux
Sistema / versión	🔧	🔧
¿Real, VM o WSL?	🔧	🔧
Modelo de CPU	🔧	🔧
Sockets	🔧	🔧
Núcleos por socket	🔧	🔧
Hilos por núcleo (SMT)	🔧	🔧
Procesadores lógicos	🔧	🔧
Nodos NUMA	🔧	🔧
Compilador y versión	🔧	🔧

> Si el equipo Linux es una VM con pocos LP (el heredado tenía 2 vCPU), la prueba de paralelismo queda limitada. Documentarlo o usar WSL/equipo real.

7. Responsabilidades y entregables

Persona	Rol	Entrega al final de la Fase 0
P1	Solución y carga	Secciones 2 y 3 aprobadas
P2	Equivalencia en Python	Confirmación de kernel, partición y workload de Python
P3	Diseño experimental	Secciones 4, 5 y 6 aprobadas
P4	Problema e hipótesis	Sección 1 aprobada; consolida el documento

Estructura del repositorio.

```text
docs/00_definicion.md
cpp/            (main.cpp, README.md)
python/         (código y README.md)
pruebas/{linux,windows}/   (scripts, procedimiento.md, resultados/)
informe/
```

8. Riesgos previstos

Riesgo	Mitigación
El compilador optimiza la carga	`-O0` + `volatile`; verificar que duplicar el workload duplica el tiempo
VM con pocos LP	Documentar la limitación; usar WSL o equipo real
`fork` no existe en Windows	`CreateProcess` relanzando el `.exe`
Ruido de otros procesos	Protocolo de la sección 4; repeticiones y desviación
Python demasiado lento	Workload reducido y documentado
Compilador antiguo en Windows	Documentar versión; preferir MSYS2/GCC reciente

---

9. Checklist de aprobación

- [ ] P4: planteamiento del problema e hipótesis (sección 1).
- [ ] P1: escenarios, APIs, kernel y workload (secciones 2 y 3).
- [ ] P2: equivalencia con Python confirmada.
- [ ] P3: matriz, protocolo, métricas y tabla de equipos (secciones 4–6).
- [ ] Los valores 🔧 fueron completados.
- [ ] Los cuatro integrantes aprueban:

Integrante	Aprobado (fecha)
P1	
P2	
P3	
P4	